

SECURE CODING REVIEW REPORT

CodeAlpha Cyber Security Internship

Task Name: Secure Coding Review

Intern Name: Muskan Raj

Internship Organization: CodeAlpha

Submission Date: 24-06-2026

1. Executive Summary

The purpose of this Secure Coding Review is to identify security vulnerabilities in a sample Python authentication program and provide recommendations to improve its security. The review focuses on common coding weaknesses that could lead to unauthorized access, data exposure, and security breaches.

2. Objective

The objective of this project is to:

- Analyze source code for security vulnerabilities.
- Identify insecure coding practices.
- Assess potential security risks.
- Recommend secure coding techniques and best practices.

3. Code Under Review

```
username = input("Enter Username: ")
password = input("Enter Password: ")

if username == "admin" and password == "admin123":
    print("Login Successful")
else:
    print("Login Failed")
```

4. Security Assessment

The reviewed code contains multiple security weaknesses that could be exploited by attackers.

Vulnerability 1: Hardcoded Credentials

Description:

The username and password are directly embedded within the source code.

Risk Level:

High

Impact:

If an attacker gains access to the source code, the credentials become immediately visible, resulting in unauthorized access.

Vulnerability 2: Weak Password Policy

Description:

The password "admin123" is simple and predictable.

Risk Level:

High

Impact:

Weak passwords are vulnerable to brute-force attacks and dictionary-based password guessing techniques.

Vulnerability 3: Plain Text Password Handling

Description:

Passwords are stored and compared in plain text format.

Risk Level:

Critical

Impact:

Sensitive authentication data may be exposed if the code or storage system is compromised.

Vulnerability 4: Lack of Input Validation

Description:

User input is accepted without validation or sanitization.

Risk Level:

Medium

Impact:

Improper input handling can increase the risk of unexpected behavior and potential security vulnerabilities.

5. Risk Analysis

Vulnerability	Risk Level
Hardcoded Credentials	High
Weak Password	High
Plain Text Password Storage	Critical
Lack of Input Validation	Medium

6. Recommendations

The following recommendations are proposed to improve application security:

1. Remove Hardcoded Credentials

Authentication credentials should never be stored directly within source code. Secure storage mechanisms such as databases or environment variables should be used.

2. Implement Strong Password Policies

Passwords should contain:

- Minimum 12 characters
- Uppercase letters
- Lowercase letters
- Numbers
- Special characters

3. Use Password Hashing

Passwords should be stored using secure hashing algorithms such as:

- bcrypt
- Argon2
- PBKDF2

4. Validate User Input

All user input should be validated and sanitized before processing.

5. Implement Multi-Factor Authentication (MFA)

MFA adds an additional security layer and reduces the risk of unauthorized access.

6. Conduct Regular Security Reviews

Periodic code reviews and security testing should be performed to identify and address vulnerabilities.

7. Conclusion

The secure coding review identified several security weaknesses in the authentication program, including hardcoded credentials, weak password usage, plain text password handling, and insufficient input validation. Implementing the recommended security controls will significantly improve the application's security posture and help protect against common cyber threats.

8. References

1. OWASP Secure Coding Practices
2. OWASP Top 10 Security Risks
3. NIST Cybersecurity Framework
4. Python Security Best Practices